

Deployment & Installation Guide

Enterprise Autonomous Market Intelligence & Competitive Strategy Platform

Prepared by: Muhammad Hassan Tariq

Table of Content

Table of Content	2
1. Introduction	3
2. Purpose of the Deployment Guide	3
3. System Requirements	3
3.1 Hardware Requirements	3
3.2 Software Requirements	3
3.3 Network Requirements	4
4. Prerequisites	4
5. Required Software	4
6. Required Python Libraries	4
7. Project Structure	5
8. Environment Configuration	5
8.1 Creating the .env File	5
8.2 Configuring Environment Variables	5
9. Database Setup	6
9.1 PostgreSQL Database Configuration	6
9.2 Neo4j Database Configuration	6
9.3 Redis Configuration	7
10. Installing Project Dependencies	7
11. Running the Application	8
11.1 Starting PostgreSQL	8
11.2 Starting Neo4j	8
11.3 Starting Redis	8
11.4 Running the FastAPI Backend	8
11.5 Opening the Executive Dashboard	8
12. Running the Live Data Pipeline	8
12.1 Executing the Live Update Cycle	8
12.2 Automating Updates with Windows Task Scheduler	8
13. Verifying the Deployment	9
14. Troubleshooting	9
15. Frequently Asked Questions (FAQ)	10
16. Conclusion	11

1. Introduction

This document provides complete instructions for installing, configuring, and running the Market Intelligence & Competitive Strategy Platform. It is intended for anyone setting up the platform on a new machine. The platform is a Python-based backend system (FastAPI) supported by three databases (PostgreSQL, Neo4j, and Redis), a set of AI reasoning agents powered by a hosted large language model (Groq), and a lightweight browser-based dashboard. It also includes an automated live-data ingestion pipeline that keeps its knowledge base current without manual intervention.

2. Purpose of the Deployment Guide

This guide exists to make the platform reproducible on any properly prepared Windows machine, without requiring prior familiarity with the codebase. It documents every prerequisite, every configuration step, and every command needed to bring the system from a blank environment to a fully running platform including the recurring live-data update process. It also documents common setup failures encountered during development and how to resolve them, so a new installer does not have to rediscover the same fixes.

3. System Requirements

3.1 Hardware Requirements

- A modern multi-core processor (any machine capable of running Windows 10/11 comfortably is sufficient)
- Minimum 8 GB RAM (16 GB recommended, since Neo4j, PostgreSQL, Redis, and the Python backend all run simultaneously alongside embedding/LLM-related processing)
- At least 10 GB free disk space (for databases, Python packages, and the Prophet/Stan build toolchain)
- Internet connectivity (required for live data ingestion, Groq API calls, and initial package/model downloads)

3.2 Software Requirements

- Windows 10 or 11
- Python 3.11 (or compatible 3.x version)
- Neo4j Desktop
- PostgreSQL 18
- Memurai (Redis-compatible server for Windows)
- A modern web browser (for the Executive Dashboard and Neo4j Browser)

3.3 Network Requirements

- Outbound internet access on standard HTTPS (port 443) for:
 - NewsAPI (news articles, industry publications, public reports, reviews)
 - SEC EDGAR (financial filings)
 - Google Patents' public search endpoint
 - Groq API (LLM reasoning)
 - Hugging Face (first-time download of the Sentence Transformers embedding model)
- Local network access on default database ports: PostgreSQL (5432), Neo4j Bolt (7687), Neo4j Browser (7474), Redis/Memurai (6379)
- No inbound network access is required; the platform runs entirely on localhost

4. Prerequisites

Before installation, ensure the following accounts and credentials are available:

- i. A Groq API key (free tier available), required for all LLM-based reasoning agents (SWOT, Strategy Advisor, Scenario Simulation, Executive Copilot)
- ii. A NewsAPI key (free tier available), required for live data ingestion (articles, industry publications, public reports, reviews)
- iii. Administrator access on the local machine, required to install PostgreSQL, Neo4j Desktop, Memurai, and to configure system Environment Variables

5. Required Software

- i. Python
- ii. PostgreSQL
- iii. Neo4j
- iv. Memurai

6. Required Python Libraries

- i. FastAPI
- ii. Uvicorn
- iii. spaCy
- iv. TextBlob
- v. Prophet

- vi. LangChain
- vii. LangGraph
- viii. · ChromaDB
- ix. · Sentence Transformers

7. Project Structure

```
Enterprise_Autonomous_Market_Intelligence_and_Competitive_Strategy_Platform/  
|  
|---- backend/  
|---- frontend/  
|---- scripts/  
|---- requirements.txt  
|---- README.md
```

8. Environment Configuration

8.1 Creating the .env File

Create a file named **.env** inside the backend folder. This file is not included in the project by default (for security) and must be created manually.

8.2 Configuring Environment Variables

Populate **backend/.env** with the following:

```
NEO4J_URI=neo4j://127.0.0.1:7687  
NEO4J_USER=neo4j  
NEO4J_PASSWORD=your_neo4j_password  
  
POSTGRES_HOST=localhost  
POSTGRES_PORT=5432  
POSTGRES_DB=marketintel_raw  
POSTGRES_USER=postgres  
POSTGRES_PASSWORD=your_postgres_password  
  
GROQ_API_KEY=your_groq_api_key  
NEWSAPI_KEY=your_newsapi_key
```

No separate configuration is required for Redis/Memurai, as the platform connects to it on its default local host and port.

9. Database Setup

9.1 PostgreSQL Database Configuration

- i. Install PostgreSQL 18, setting a password for the **postgres** superuser during installation.
- ii. Open a terminal and connect:

```
psql -U postgres
```

- iii. Create the database:

```
CREATE DATABASE marketintel_raw;  
\c marketintel_raw
```

- iv. Create the two required tables:

```
CREATE TABLE raw_articles (  
    id SERIAL PRIMARY KEY,  
    source TEXT,  
    url TEXT,  
    title TEXT,  
    content TEXT,  
    published_date DATE,  
    company_mentioned TEXT,  
    processed BOOLEAN DEFAULT FALSE,  
    created_at TIMESTAMP DEFAULT NOW()  
);  
  
CREATE TABLE raw_reviews (  
    id SERIAL PRIMARY KEY,  
    product_name TEXT,  
    source TEXT,  
    review_text TEXT,  
    rating FLOAT,  
    review_date DATE,  
    processed BOOLEAN DEFAULT FALSE,  
    created_at TIMESTAMP DEFAULT NOW()  
);
```

- v. Verify the tables were created:

```
\dt
```

9.2 Neo4j Database Configuration

- i. Install Neo4j Desktop.
- ii. Create a new local DBMS (e.g. named **marketintel-db**) and start it.
- iii. Confirm its connection URI (default **neo4j://127.0.0.1:7687**) and note the credentials for the **.env** file.

- iv. Populate the graph with initial data:

```
python scripts\setup_knowledge_graph.py
python scripts\enrich_knowledge_graph.py
python scripts\expand_companies.py
python scripts\expand_events.py
```

9.3 Redis Configuration

- i. Install Memurai (Developer edition), it registers itself as a Windows service and starts automatically on the default Redis port (6379).
- ii. Verify it is running:

```
"C:\Program Files\Memurai\memurai-cli.exe" ping
```

- iii. Expected response: **PONG**

10. Installing Project Dependencies

- i. Create and activate a Python virtual environment inside the **backend** folder:

```
cd backend
python -m venv venv
venv\Scripts\activate
```

- ii. Install all required packages:

```
pip install fastapi uvicorn neo4j psycpg2-binary python-dotenv redis
pip install spacy textblob chromadb sentence-transformers
pip install prophet cmdstanpy pandas
pip install langchain langgraph langchain-groq
pip install networkx requests
python -m spacy download en_core_web_sm
python -m textblob.download_corpora
```

- iii. Install Prophet's Stan backend build toolchain (Windows-specific):

```
python -m cmdstanpy.install_cxx_toolchain
python -m cmdstanpy.install_cmdstan --overwrite
```

If the build fails to find **mingw32-make**, add both of the following folders to the system PATH (ahead of any pre-existing MinGW installation), then open a new terminal:

```
C:\Users\<your-username>\.cmdstan\RTools40\usr\bin
C:\Users\<your-username>\.cmdstan\RTools40\mingw64\bin
```

- iv. Build the initial semantic search index:

```
python scripts\build_rag_index.py
```

11. Running the Application

11.1 Starting PostgreSQL

PostgreSQL runs automatically as a Windows service once installed. No manual start is required.

11.2 Starting Neo4j

Open Neo4j Desktop and click **Start** on the **marketintel-db** instance. Wait until its status shows **Active**.

11.3 Starting Redis

Memurai runs automatically as a Windows service once installed. No manual start is required.

11.4 Running the FastAPI Backend

```
cd market-intel-platform\backend
venv\Scripts\activate
uvicorn app.main:app --reload --port 8000
```

Leave this terminal window open. The API is now available at **http://127.0.0.1:8000**, with interactive documentation at **http://127.0.0.1:8000/docs**.

11.5 Opening the Executive Dashboard

With the backend running, open **frontend/dashboard.html** directly in a web browser (double-click the file, no server required for the dashboard itself).

12. Running the Live Data Pipeline

12.1 Executing the Live Update Cycle

The platform includes a combined script that fetches and processes live data from all six implemented source types (news articles, industry publications, public-report coverage, SEC financial filings, patent data, and customer reviews) in a single run:

```
python scripts\live_update_cycle.py
```

This can be run manually at any time to refresh the platform's data.

12.2 Automating Updates with Windows Task Scheduler

To run the update cycle automatically:

1. Open **Task Scheduler** and select **Create Task**.
2. On the **General** tab, name it (e.g. "Market Intel Live Update").
3. On the **Triggers** tab, create a new trigger set to repeat every **6 hours**, indefinitely.
4. On the **Actions** tab, set:
 1. Program/script: the full path to the virtual environment's **python.exe** (e.g. **C:\...\backend\venv\Scripts\python.exe**)
 2. Add arguments: **scripts\live_update_cycle.py**
 3. Start in: the project's root folder
5. On the **Conditions** tab, uncheck "Start the task only if the computer is on AC power."

6. Save the task.

Note: the 6-hour interval reflects NewsAPI's free-tier daily request quota; a shorter interval risks exceeding that quota once multiple companies and source types are being polled per cycle.

13. Verifying the Deployment

After completing setup, verify each layer independently:

1. **Neo4j:** In Neo4j, run **MATCH (n) RETURN labels(n), count(*)** should show non-zero counts for Company, Product, Event, Article, and Review nodes.
2. **PostgreSQL:** **SELECT count(*) FROM raw_articles;** and **SELECT count(*) FROM raw_reviews;** should return non-zero counts after running the live update cycle at least once.
3. **Redis:** "**C:\Program Files\Memurai\memurai-cli.exe**" **ping** should return **PONG**.
4. **Backend API:** Visiting **http://127.0.0.1:8000/docs** should show the full list of endpoints.
5. **Dashboard:** Opening **frontend/dashboard.html** should display populated charts and tables, not blank panels.
6. **AI agents:** A test call to **http://127.0.0.1:8000/api/swot/Samsung** (or any tracked company) should return a JSON response with "**status**": "**ok**".

14. Troubleshooting

Symptom	Cause	Resolution
neo4j.exceptions.ServiceUnavailable	Neo4j database not started	Start the database in Neo4j Desktop
psql or redis-cli not recognized	Tool's install folder not on system PATH	Add the folder to PATH via Environment Variables; open a new terminal
AttributeError: 'Prophet' object has no attribute 'stan_backend'	Stan build toolchain missing or PATH conflict with an older MinGW install	Install the RTools toolchain and ensure its folders precede any other MinGW entry in PATH
429 Too Many Requests from NewsAPI	Free-tier daily quota (100 requests) exceeded	Reduce the Task Scheduler frequency, or wait for the daily quota reset
Duplicate nodes in Neo4j after rerunning a load script	A Cypher query used CREATE instead of MERGE	Use MERGE keyed on a unique identifier for any script that may be rerun
Dashboard shows blank charts	Backend not running, or CORS not enabled in main.py	Confirm the backend is running; confirm CORSMiddleware is configured
fetch_live_articles.py crashes partway through	A single company's request failed (network issue) and was not caught	Ensure each company's fetch is wrapped in its own try/except so one failure does not stop the rest

15. Frequently Asked Questions (FAQ)

Q1. Do I need to complete the installation every time I use the platform?

Answer:

No. Installation and configuration are one-time tasks. For daily use, simply start the required services (Neo4j, PostgreSQL, and Memurai if necessary), run the FastAPI backend, and open the Executive Dashboard.

Q2. What should I do if the Executive Dashboard does not display any data?

Answer:

Ensure that:

- The FastAPI backend is running.
- PostgreSQL and Neo4j are active.
- The required data has been loaded into the databases.
- The browser is refreshed after the backend starts.

Q3. Is an internet connection required?

Answer:

Yes, An internet connection is required for live data ingestion, API communication (Groq and NewsAPI), downloading AI models during initial setup, and retrieving the latest market information.

Q4. How can I update the platform with the latest market data?

Answer:

Run the live update cycle manually using the provided script or configure Windows Task Scheduler to execute the update automatically every six hours.

Q5. Can I change the live update interval?

Answer:

Yes. The update frequency can be modified in Windows Task Scheduler. However, shorter intervals may exceed the free API request limits of NewsAPI.

Q6. What should I do if Neo4j or PostgreSQL fails to connect?

Answer:

Verify that the database services are running, check the connection details in the .env file, and ensure the configured ports are correct.

Q7. Where are the raw articles and customer reviews stored?

Answer:

Raw articles and customer reviews are stored in the PostgreSQL database before being processed by the NLP pipeline and integrated into the Knowledge Graph.

Q8. Which web browser is recommended for using the dashboard?

Answer:

The Executive Dashboard works with modern web browsers. Google Chrome and Microsoft Edge are recommended for the best performance and compatibility.

Q9. Can the platform be deployed on another computer?

Answer:

Yes. Install all required software, configure the environment variables, install the project dependencies, and follow the deployment steps described in this guide.

Q10. How can I verify that the deployment was successful?

Answer:

Successful deployment can be verified by:

- Confirming PostgreSQL, Neo4j, and Memurai are running.
- Opening the FastAPI API documentation (<http://127.0.0.1:8000/docs>).
- Opening the Executive Dashboard.
- Running the live update cycle successfully.
- Confirming that dashboard charts and AI-generated insights are displayed correctly.

16. Conclusion

This Deployment & Installation Guide provides the complete procedure for installing, configuring, and running the **Enterprise Autonomous Market Intelligence & Competitive Strategy Platform**. It covers the required software, system configuration, database setup, dependency installation, application startup, live data pipeline execution, deployment verification, troubleshooting, and frequently asked questions.

By following the instructions provided in this guide, users can successfully deploy the platform on a Windows environment and operate all major components, including the FastAPI backend, PostgreSQL database, Neo4j Knowledge Graph, Redis cache, AI modules, and the Executive Dashboard.

After successful deployment, the platform is capable of collecting market intelligence data, processing information through AI-powered components, generating strategic insights, and presenting business analytics through an interactive dashboard. Proper deployment and regular execution of the live update cycle help ensure that the platform remains up to date and continues to provide accurate and reliable market intelligence for analysis and decision-making.